CaloriX — Project Documentation

CaloriX is a full-stack calorie and nutrition tracking application. It combines a FastAPI backend, a React (Vite) frontend, a scikit-learn based ML food-classification engine, and Google Gemini–powered AI features (meal suggestions, food-photo analysis, and a conversational nutrition coach with tool-calling).

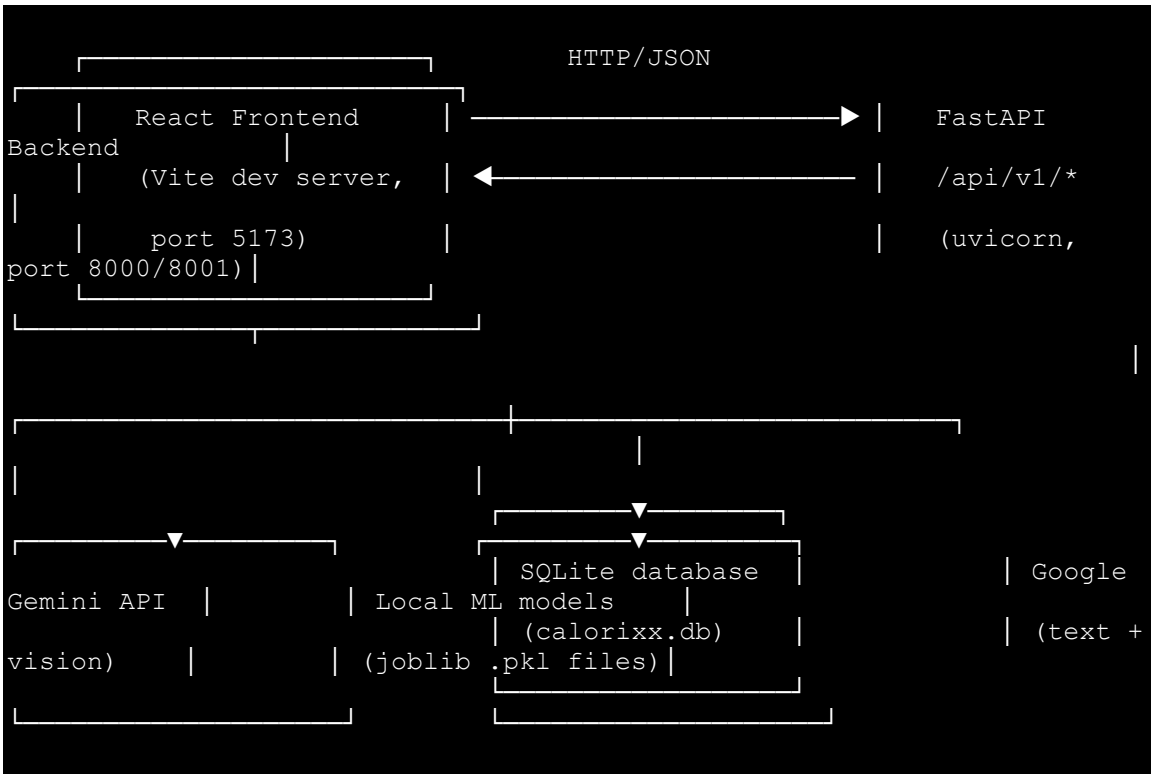
Table of Contents

1. Tech Stack
2. High-Level Architecture
3. Backend
    - Project Layout
    - Configuration
    - Authentication
    - Database Models
    - API Reference
    - Services
    - AI & ML Subsystems
4. Frontend
    - Project Layout
    - Routing
    - State & Auth
    - Services (API Clients)
    - Key Pages & Components
5. Setup & Installation
6. Environment Variables
7. Known Gaps / Notes for Contributors

Tech Stack

Layer	Technology
Backend framework	FastAPI
ORM / DB	SQLAlchemy 2.0 (Mapped/mapped_column style) + SQLite (`calorixx.db`)
Auth	JWT (access + refresh tokens) via `python-jose`, password hashing via `passlib`/bcrypt
AI (LLM)	Google Gemini (`google-generativeai`), model `gemini-2.5-flash`
ML	scikit-learn (classifier, scaler, label encoder, nearest-neighbor recommender), pandas, joblib
Frontend framework	React 18 + Vite
Routing	React Router v6
Styling	Tailwind CSS (custom dark "CaloriX" design tokens)
HTTP client	Axios
Markdown rendering	`react-markdown` (used in the AI Chat UI)
Icons	`lucide-react`

High-Level Architecture



The frontend talks exclusively to the FastAPI backend via `axios` (`frontend/src/services/api.js`), which is configured with base URL `http://localhost:8001/api/v1` and attaches a bearer token from `localStorage` (`cx_token`) to every request. A 401 response clears the token and redirects to `/login`.

Backend

Backend Project Layout

```
    app/
    ├── main.py                      # FastAPI app factory, CORS,
startup table creation
    ├── api/
    │   ├── router.py                 # Aggregates all v1 routers under
/api/v1
    │   └── v1/
    │       ├── auth.py               # /auth  - register, login, me
    │       ├── health.py             # /health
    │       ├── profile.py            # /profile - BMR/TDEE/macro
profile
    │       ├── meals.py              # /meals - CRUD meal logging
    │       ├── ml.py                 # /ml - food classification,
similarity, search
    │       ├── ai.py                 # /ai - Gemini meal suggestions &
image analysis
    │       ├── metrics.py            # /metrics - dashboard analytics
    │       └── ai_chat.py            # /ai-chat - chat sessions +
coach messaging
    ├── auth/
    │   └── dependencies.py           # get_current_user,
get_optional_ai_user
    ├── core/
    │   ├── config.py                 # Settings (pydantic-settings)
    │   └── security.py               # password hashing, JWT
create/decode
    ├── database/
    │   └── session.py                # SQLAlchemy engine/session/Base
    ├── models/                       # SQLAlchemy ORM models
    │   ├── user.py
    │   ├── user_profile.py
    │   ├── meal.py
    │   └── chat.py                   # ChatSession, ChatMessage
    ├── schemas/                      # Pydantic request/response
schemas
    │   ├── user.py
    │   ├── profile.py
    │   ├── meal.py
    │   ├── ml.py
    │   ├── ai.py
    │   ├── chat.py
    │   └── metrics.py
    ├── services/                     # Business logic layer
    │   ├── user_service.py
    │   ├── profile_service.py
    │   ├── meal_service.py
    │   ├── ml_service.py
    │   ├── ai_service.py
    │   ├── ai_tools.py                # Gemini function-calling tool
schemas + dispatcher
    │   ├── chat_service.py
```

```
    │   └── metrics_service.py
    └── ml/                              # Trained model artifacts
(loaded at startup)
        ├── metadata.json
        ├── best_meal_classifier.pkl
        ├── feature_scaler.pkl
        ├── label_encoder.pkl
        ├── food_recommender.pkl
        └── CaloriX_Nutrition_Dataset_Clean.csv
```

Configuration

`app/core/config.py` defines a `Settings` (pydantic-settings) class, cached via `@lru_cache`, loaded from environment variables / `.env`:

Setting	Default	Purpose
`APP_NAME`	`CaloriX`	App title
`APP_VERSION`	`0.1.0`	App version shown in `/docs`
`DEBUG`	`False`	Enables SQL echo
`API_PREFIX`	`/api/v1`	(declared but the router itself hardcodes `/api/v1`)
`DATABASE_URL`	`sqlite:///./calorixx.db`	SQLAlchemy connection string
`SECRET_KEY`	placeholder — **must be changed in production**	JWT signing secret
`ALGORITHM`	`HS256`	JWT signing algorithm
`ACCESS_TOKEN_EXPIRE_MINUTES`	`30`	Access token lifetime
`REFRESH_TOKEN_EXPIRE_DAYS`	`7`	Refresh token lifetime
`CORS_ORIGINS`	`["http://localhost:5173"]`	Allowed frontend origins
`GEMINI_API_KEY`	`""`	Enables all AI features when set
`REQUIRE_AI_AUTH`	`False`	If `True`, `/ai/*` endpoints require a valid JWT

The app creates all database tables on startup via `Base.metadata.create_all(bind=engine)` (no Alembic migrations are wired up yet).

Authentication

- **Registration** (`POST /api/v1/auth/register`): creates a `User` with a bcrypt-hashed password; rejects duplicate email/username with `409`.
- **Login** (`POST /api/v1/auth/login`): validates credentials, returns `{ access_token, refresh_token, token_type }`.
- **Current user** (`GET /api/v1/auth/me`): requires `Authorization: Bearer <access_token>`.
- Tokens are decoded via `app.core.security.decode_token`; the `sub` claim holds the user ID.
- `get_current_user` (in `auth/dependencies.py`) is the standard FastAPI dependency used on nearly all protected routes.
- `get_optional_ai_user` allows anonymous access to AI endpoints unless `REQUIRE_AI_AUTH=True`, in which case a valid bearer token is mandatory.

Database Models

`User` (`users`)

- `id`, `email` (unique), `username` (unique), `hashed_password`, `is_active`, `created_at`, `updated_at`
- 1:1 with `UserProfile` (cascade delete), 1:many with `Meal` (cascade delete)

`UserProfile` (`user_profiles`)

- Inputs: `age`, `gender`, `height_cm`, `weight_kg`, `activity_level`, `goal`
- Calculated: `bmr`, `tdee`, `daily_calorie_target`, `protein_target_g`, `fat_target_g`, `carb_target_g`
- One profile per `user_id` (unique FK)

`Meal` (`meals`)

- `food_name`, `calories`, `protein`, `carbs`, `fat`, `meal_type` (Breakfast/Lunch/Dinner/Snack), `meal_date`
- `food_category` (optional), `is_ai_predicted` (bool) — set when the ML classifier supplied the category
- Belongs to a `User` (cascade delete)

`ChatSession` (`chat_sessions`) / **`ChatMessage`** (`chat_messages`)

- A session has a `title`, an evolving `summary` (used to compress old history), and many ordered messages
- Each message has `role` (`user` or `model`) and `content`

API Reference

All routes are mounted under `/api/v1`. Interactive docs are auto-generated at `/docs` (Swagger) and `/redoc`.

Health

Method	Path	Auth	Description
GET	`/health`	none	Liveness check → `{status, service}`

Auth (`/auth`)

Method	Path	Auth	Description
POST	`/auth/register`	none	Create account
POST	`/auth/login`	none	Get access/refresh tokens
GET	`/auth/me`	Bearer	Current user profile

Profile (`/profile`)

Method	Path	Auth	Description
GET	`/profile`	Bearer	Get saved profile + calculated macros (404 if not set up)
POST	`/profile`	Bearer	Create/update profile; recalculates BMR/TDEE/targets

Meals (`/meals`)

Method	Path	Auth	Description
POST	`/meals`	Bearer	Log a new meal
GET	`/meals?target_date=YYYY-MM-DD`	Bearer	Meals + daily summary for a date
PUT	`/meals/{meal_id}`	Bearer	Partial update of a meal
DELETE	`/meals/{meal_id}`	Bearer	Delete a meal

ML (`/ml`)

Method	Path	Auth	Description
POST	`/ml/predict`	Bearer	Predict a food's category from nutrition features
POST	`/ml/similar-foods`	Bearer	k-NN nearest foods by nutrition profile + text similarity
GET	`/ml/foods?q=&limit=`	Bearer	Substring search over the nutrition dataset

AI (`/ai`) — Gemini-powered

Method	Path	Auth	Description
POST	`/ai/suggest-meals`	Optional*	Generate 3–5 meal ideas from available ingredients + user macro context
POST	`/ai/analyze-image`	Optional*	Vision analysis of a food photo → detected items, macros, confidence

* Optional unless `REQUIRE_AI_AUTH=True`. Both return `503` if `GEMINI_API_KEY` is not configured.

Metrics (`/metrics`)

Method	Path	Auth	Description
GET	`/metrics?range=7D|30D|90D`	Bearer	Consolidated dashboard payload: stat cards, weekly calorie chart, macro distribution, goal progress, insights, recent activity

AI Chat (`/ai-chat`)

Method	Path	Auth	Description
GET	`/ai-chat/sessions`	Bearer	List chat sessions (most recently updated first)
POST	`/ai-chat/sessions`	Bearer	Create a session (`{title}`)
GET	`/ai-chat/sessions/{id}`	Bearer	Session detail incl. all messages
DELETE	`/ai-chat/sessions/{id}`	Bearer	Delete a session
PUT	`/ai-chat/sessions/{id}/title`	Bearer	Rename a session
POST	`/ai-chat/sessions/{id}/message`	Bearer	Send a user message; returns the AI's reply

Services

- **`UserService`** — registration (duplicate checks), authentication, token issuance.
- **`ProfileService`** — Mifflin-St Jeor BMR calculation, TDEE via activity multiplier, calorie-target adjustment by goal (Weight Loss = TDEE-500 floor 1200 kcal; Muscle Gain = TDEE+300; Maintenance = TDEE), and macro split (protein 2 g/kg capped at 45% kcal, fat 25% kcal, carbs = remainder).
- **`MealService`** — CRUD for meals plus `get_daily_summary` (totals vs. profile targets, remaining calories).
- **`MetricsService`** — aggregates meals over a date range into stat cards, a 7-day calorie chart, macro % distribution, a naive "goal progress" percentage, textual insights, and recent activity.
- **`MLService`** — singleton that loads `metadata.json`, the trained classifier/scaler/label-encoder/recommender `.pkl` files, and the nutrition CSV at import time; exposes `predict`, `get_similar_foods` (hybrid numeric + TF-IDF nearest neighbors), and `search_foods`.

- **`AIService`** — wraps Gemini for (1) ingredient-based meal suggestions, (2) food-image analysis, and (3) the tool-calling chat coach (`chat_with_coach`). Builds rich user context (profile, today's meals, last 10 meals, 7-day nutrition history) via `_get_user_context`.
- **`ChatService`** / **`PromptBuilder`** — manages chat sessions, builds the system + context + history prompt sent to Gemini, and auto-summarizes older messages (beyond `MAX_HISTORY = 10`) using Gemini itself to keep prompts small.

AI & ML Subsystems

Meal suggestions & image analysis (`ai_service.py`)

- Uses `gemini-2.5-flash` for both text and vision.
- Prompts instruct the model to return **strict JSON**; responses are stripped of Markdown code fences and parsed, then clamped/validated field-by-field before being returned.
- User context (remaining calories/macros, goal, ingredient list) is injected into the meal-suggestion prompt.

AI Coach / Function Calling (`ai_service.chat_with_coach` + `ai_tools.py`)

- Tools are only enabled when the latest user message looks like a mutation request (heuristic keyword match: "add", "log", "delete", "update weight", etc.) — read-only chat runs without tools to save latency/cost.
- Available Gemini tools:
  - `add_meal(food_name, meal_type, calories, protein, carbs, fat, meal_date?)`
  - `delete_meal(meal_id)`
  - `update_weight(weight_kg)`
- `ToolDispatcher.dispatch` normalizes alternate argument names (`name`→`food_name`, `protein_g`→`protein`, etc.), executes the corresponding service call, and returns a plain-text result that is fed back to Gemini as a `function_response` so it can produce a natural-language confirmation.

ML food classification / recommendation (`ml_service.py`)

- Loaded once at process start (singleton pattern).
- Feature engineering derives density/ratio features (protein/fat/carb density per gram, calorie density, macro ratios, energy density) both for the static dataset and for any incoming prediction request, ensuring feature parity with training.
- `predict` → food category via the trained classifier + label encoder.
- `get_similar_foods` → combines scaled numeric features with a TF-IDF text vector (name/category/cuisine) and queries a fitted nearest-neighbors index.

- `search_foods` → simple case-insensitive substring match on `food_name` over the CSV dataset.

Frontend

Frontend Project Layout

```
    src/
    ├── main.jsx                # App bootstrap: StrictMode,
BrowserRouter, AuthProvider
    ├── App.jsx                 # Route table
    ├── index.css               # Tailwind + CaloriX dark theme design
tokens
    ├── layouts/
    │   └── MainLayout.jsx       # Header/nav/footer shell (Outlet)
    ├── context/
    │   └── AuthContext.jsx      # Auth state, login/register/logout,
token persistence
    ├── components/
    │   ├── ProtectedRoute.jsx    # Redirects unauthenticated users to
/login
    │   ├── GuestRoute.jsx        # Redirects authenticated users away
from /login,/register
    │   └── ai/
    │       ├── IngredientInput.jsx
    │       ├── MealSuggestionCard.jsx
    │       └── ImageAnalyzerCard.jsx
    ├── pages/
    │   ├── Home.jsx
    │   ├── Login.jsx
    │   ├── Register.jsx
    │   ├── Dashboard.jsx
    │   ├── Profile.jsx
    │   ├── MealTracker.jsx
    │   ├── Metrics.jsx
    │   ├── AIAssistant.jsx      # "Pantry Chef" + "Food Vision" tabs
    │   └── AIChat.jsx           # Conversational AI coach with
session sidebar
    └── services/                # Axios wrappers, one per backend
resource
        ├── api.js                # Base axios instance + interceptors
        ├── authService.js
        ├── profileService.js
        ├── mealService.js
        ├── mlService.js
        ├── aiService.js
        ├── aiChatService.js
        └── metricsService.js
```

Routing

Defined in `App.jsx` using React Router v6, all nested under a single `MainLayout`:

Path	Access	Page
`/`	Public	`Home`
`/login`	Guest only	`Login`
`/register`	Guest only	`Register`
`/dashboard`	Protected	`Dashboard`
`/profile`	Protected	`Profile`
`/meals`	Protected	`MealTracker`
`/ai-assistant`	Protected	`AIAssistant`
`/ai-chat`	Protected	`AIChat`
`/metrics`	Protected	`Metrics`

`ProtectedRoute` shows a spinner while auth is resolving, then redirects to `/login` (preserving the intended destination in router state) if unauthenticated. `GuestRoute` does the inverse for `/login` and `/register`.

State & Auth

`AuthContext` (`context/AuthContext.jsx`):

- Persists the JWT access token in `localStorage` under `cx_token`.
- On mount / token change, hydrates the current user via `GET /auth/me`; clears the token if invalid.
- `login(email, password)` → stores token, fetches `/auth/me`.
- `register(username, email, password)` → registers, then auto-logs in.
- `logout()` → clears token/user from state and storage.

Services (API Clients)

`services/api.js` creates a shared Axios instance:

- `baseURL: http://localhost:8001/api/v1`
- Request interceptor attaches `Authorization: Bearer <cx_token>` when present.

- Response interceptor: on `401`, clears the token and hard-redirects to `/login`.

Each resource has a thin wrapper module (`mealService`, `profileService`, `mlService`, `aiService`, `metricsService`, `aiChatService`, `authService`) that simply calls the matching backend endpoint and returns `response.data`.

Key Pages & Components

- **`Dashboard`** — shows a "complete your profile" CTA if no profile exists (404 from `/profile`); otherwise renders BMR/TDEE/calorie-target cards and a macro-distribution breakdown computed client-side from the profile's macro grams.
- **`Profile`** — form for age/gender/height/weight/activity/goal; client-side range validation before submit; on success, saves via `POST /profile` and redirects to the dashboard.
- **`MealTracker`** — date-scoped meal log with:
  - Debounced food-name autocomplete backed by `mlService.searchFoods`.
  - "Auto-detect" AI category prediction (`mlService.predict`) applied either on food selection or on demand.
  - "Similar foods" AI recommendations (`mlService.getSimilarFoods`) that can quick-fill the add-meal form.
  - Grouped display by meal type with per-group calorie subtotal, plus a running daily summary and macro progress bars sourced from `GET /meals`.
- **`AIAssistant`** — two tabs:
  - *Pantry Chef*: `IngredientInput` collects ingredients/meal-type/dietary-prefs/max-calories → `aiService.suggestMeals` → renders `MealSuggestionCard`s with an "Add to Meal Tracker" action that posts to `mealService.addMeal`.
  - *Food Vision*: `ImageAnalyzerCard` (drag-and-drop upload, 5 MB client-side cap) → `aiService.analyzeImage` → displays detected items/macros/confidence with a one-click "Add to Meal Tracker".
- **`AIChat`** — session sidebar (create/select/delete/rename) + a chat pane that renders assistant responses as Markdown (`react-markdown`) and posts new messages via `aiChatService.sendMessage`, which can trigger backend tool calls (add/delete meal, update weight) transparently.
- **`Metrics`** — range-toggle (7D/30D/90D) dashboard fed entirely by `GET /metrics`: KPI cards, weekly calorie bars, macro split, goal progress, textual insights, recent activity feed.

Setup & Installation

Backend

```
    # from the backend project root
    python -m venv .venv
    source .venv/bin/activate        # Windows: .venv\Scripts\activate
    pip install -r requirements.txt  # fastapi, sqlalchemy, pydantic-
settings, python-jose,
                                      # passlib[bcrypt], google-
generativeai, scikit-learn,
                                      # pandas, joblib, uvicorn, etc.

    # create a .env (see Environment Variables below)
    cp .env.example .env

    uvicorn app.main:app --reload --port 8001
```

- Swagger UI: `http://localhost:8001/docs`
- Tables are auto-created on startup against the SQLite file configured by `DATABASE_URL` (default `calorixx.db`).
- ML model artifacts must exist under `app/ml/` (`metadata.json`, the four `.pkl` files, and the nutrition CSV) or `MLService` will raise on import — the `/ml/*` and AI-category-prediction features depend on it.

Frontend

```
    cd frontend
    npm install
    npm run dev      # Vite dev server on http://localhost:5173
```

- `vite.config.js` proxies `/api` to `http://127.0.0.1:8000` for dev convenience, **but** `src/services/api.js` currently hardcodes `http://localhost:8001/api/v1` as its `baseURL` — make sure the backend port you run matches whichever the frontend actually calls (update one or the other if you change ports).
- `npm run build` produces a production bundle; `npm run preview` serves it locally.

Environment Variables

Backend (`.env`):

```
    DATABASE_URL=sqlite:///./calorixx.db
    SECRET_KEY=replace-with-a-long-random-value   # e.g. `openssl rand
-hex 32`
    ALGORITHM=HS256
    ACCESS_TOKEN_EXPIRE_MINUTES=30
    REFRESH_TOKEN_EXPIRE_DAYS=7
    CORS_ORIGINS=["http://localhost:5173"]
    GEMINI_API_KEY=your-google-generative-ai-key
    REQUIRE_AI_AUTH=false
    DEBUG=false
```

Frontend: no `.env` is currently read by the Vite config; the API base URL is hardcoded in `src/services/api.js`.

Known Gaps / Notes for Contributors

- **Port mismatch**: `vite.config.js`'s dev proxy targets port `8000`, while `api.js` calls port `8001` directly (bypassing the proxy). Pick one convention and align both.
- **No DB migrations**: schema changes rely on `Base.metadata.create_all`, which won't alter existing tables — introducing Alembic is recommended before any production schema change.
- **`SECRET_KEY` default is a placeholder** and must be overridden outside of local development.
- **Goal-progress metric is a placeholder**: `MetricsService` computes `goal_progress.progress_pct` from calorie-target adherence, not actual weight-loss/gain progress, since `UserProfile` has no `target_weight` field yet.
- **Mutation-intent detection for the AI coach is heuristic** (keyword matching in `ai_service._is_mutation_intent`); it will enable/disable Gemini tool access based on simple substring checks and could mis-classify ambiguous phrasing.
- **Chat summarization failures are silently swallowed** (`except Exception: pass` in `ChatService.handle_message`), so if Gemini summarization fails, history simply grows unbounded until the next successful run.

- **`analyze_image` response shape drift**: the frontend's `AIAssistant.jsx` reads `analysisResult.food_name`, `.meal_type`, and `.reasoning`, but the backend's `ImageAnalysisResponse` schema doesn't define those fields (it returns `detected_food_items`, `predicted_meal_category`, `notes`, etc.) — double-check this mapping if the food-vision card seems to render blanks.